



INTRODUCTION TO SOFTWARE ENGINEERING

Part II – Software Design

Software Engineering Department
College of Information and Communication Technology

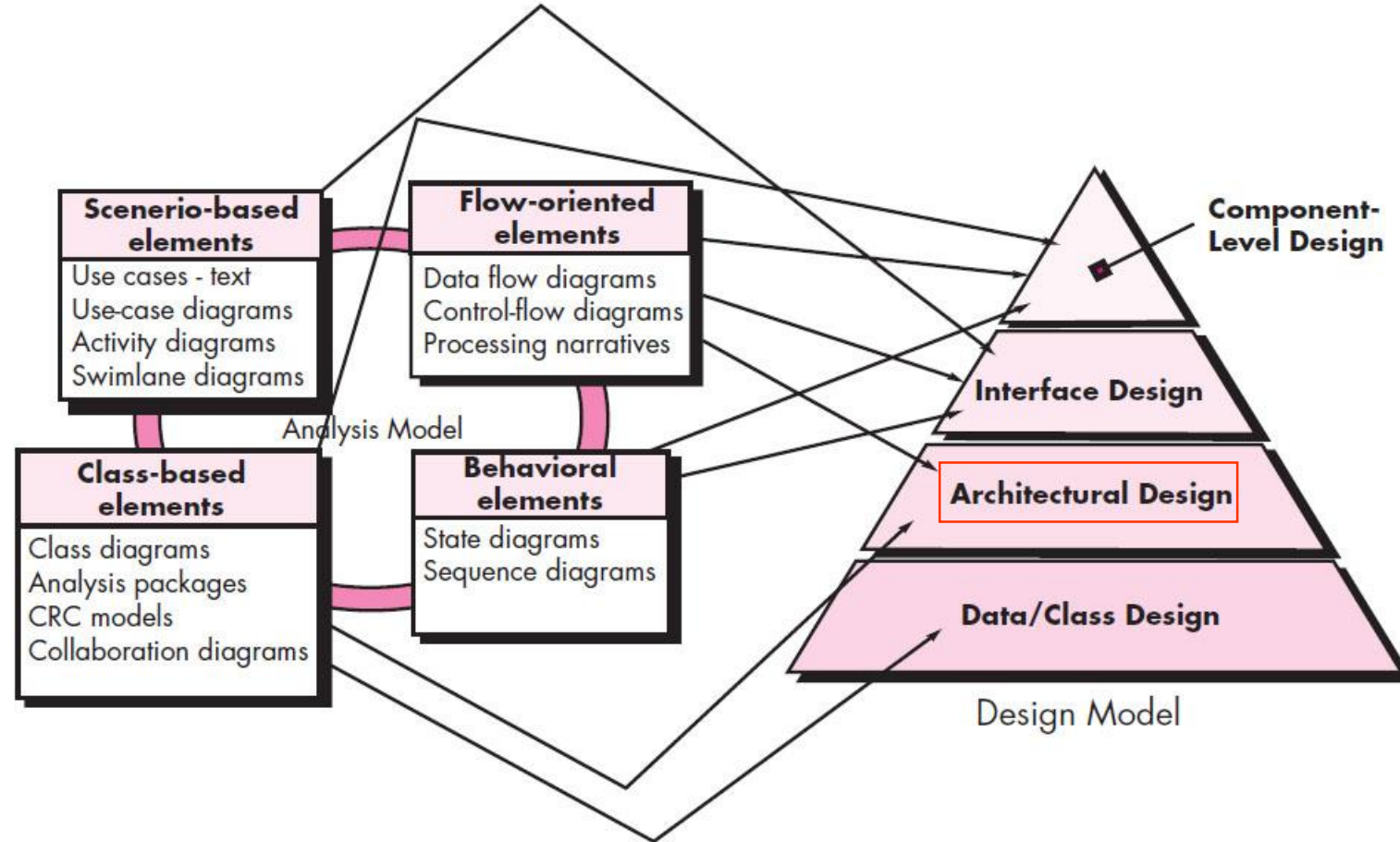
Content

- Requirements analysis & specification (Definition)
- Design
- Implementation (Programming)
- Testing
- Delivery and Maintenance

Design

- Design is the creative process of figuring out how to implement all of the customer's requirements; the resulting plan is also called the design
- Early design decisions address the system's architecture
- Later design decisions address how to implement the individual units

Content

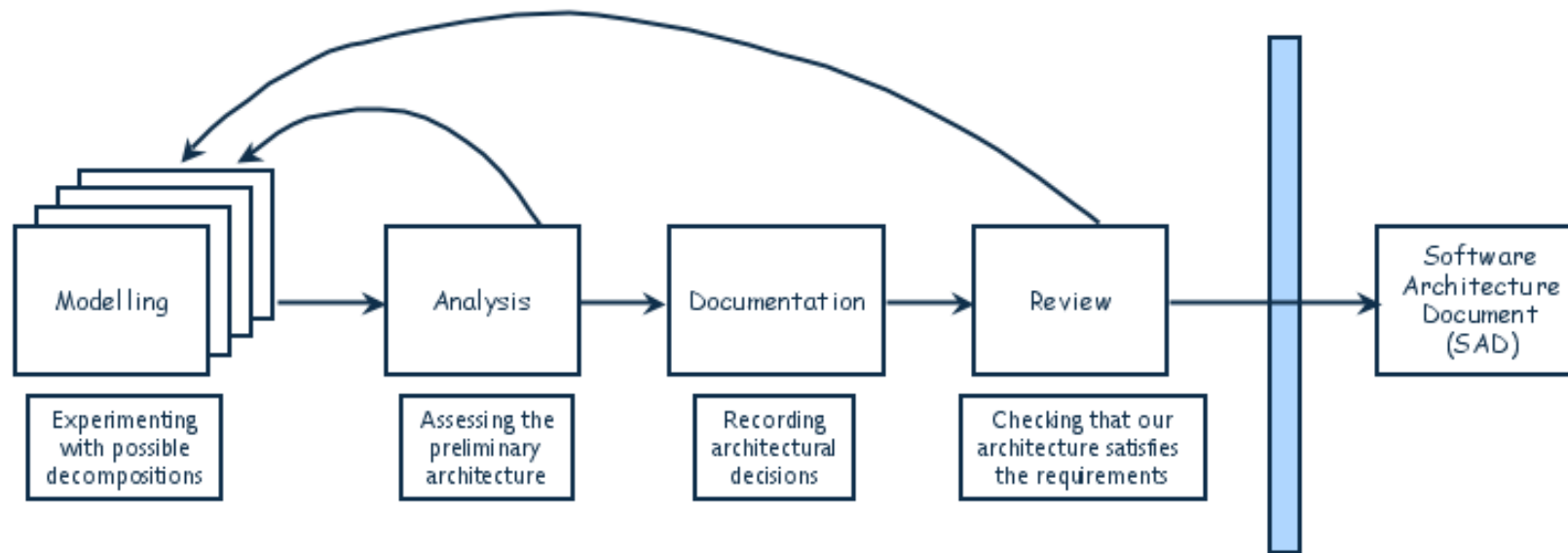


Content – Architectural Design

- The Design Process
- Modeling Architectures
- Decomposition and Views
- Popular Design Methods
- Architectural Views
- Architectural Styles and Strategies
- Collaborative Design
- Architecture Evaluation and Refinement
- Documenting Software Architectures
- Architecture Design Review

Design Process

- Designing software system is an iterative process
- The final outcome is the software architecture document (SAD)

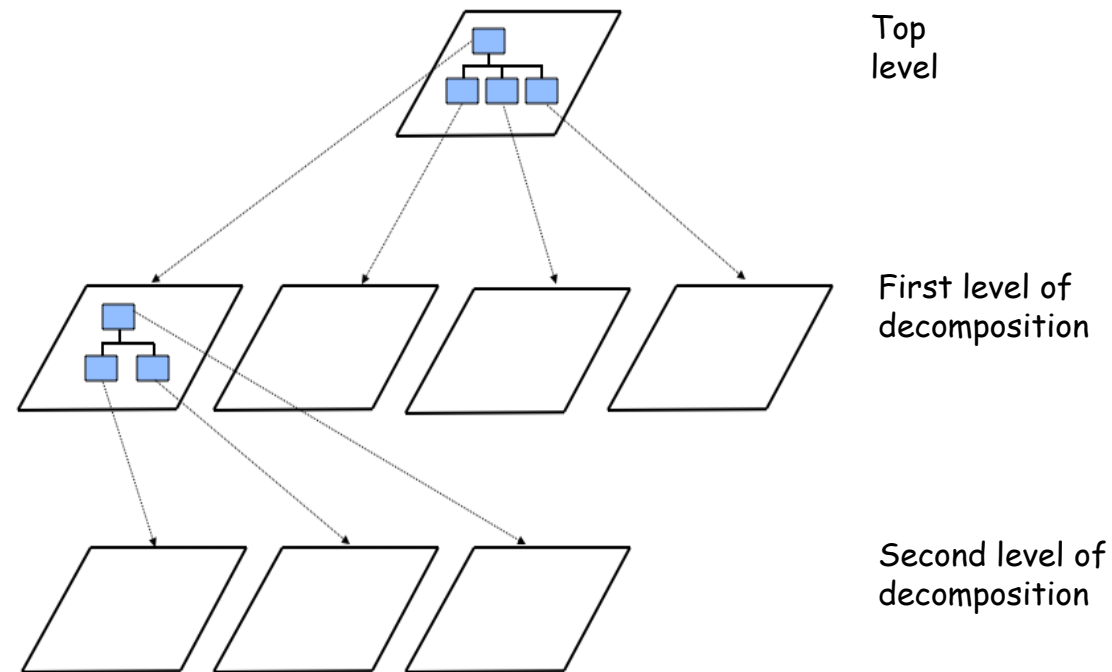


Modeling Architectures

- Collection of models helps to answer whether the proposed architecture meets the specified requirements
- Six ways to use the architectural models:
 - to understand the system
 - to determine amount of reuse from other systems and the reusability of the system being designed
 - to provide blueprint for system construction
 - to reason about system evolution
 - to analyze dependencies
 - to support management decisions and understand risks

Decomposition and Views

- High-level description of system's key elements
- Creating a hierarchy of information with increasing details



Decomposition and Views – Popular Design Methods

- Some design problems have no existing solutions
 - Designers must decompose to isolate key problems
- Some popular design methods:
 - Functional decomposition
 - Feature-oriented decomposition
 - Data-oriented decomposition
 - Process-oriented decomposition
 - Event-oriented decomposition
 - Object-oriented design

Decomposition and Views – Architectural Views

- Common types of architectural views include:
 - Decomposition view
 - Dependencies view
 - Generalization view
 - Execution view
 - Implementation view
 - Deployment view
 - Work-assignment view

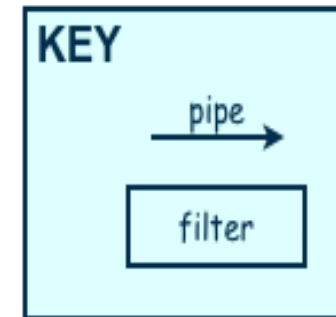
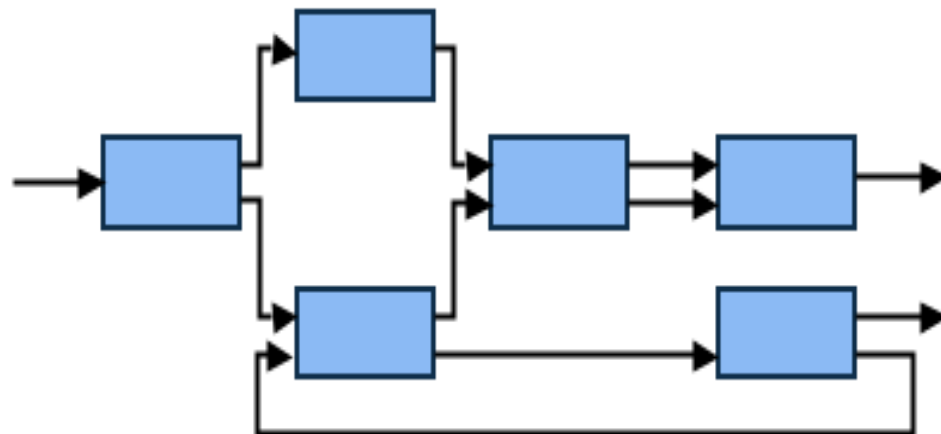
Architectural Styles and Strategies

- Pipes-and-Filter
- Client-Server
- Peer-to-Peer
- Publish-Subscribe
- Repositories
- Layering

Architectural Styles and Strategies

Pipes-and-Filter

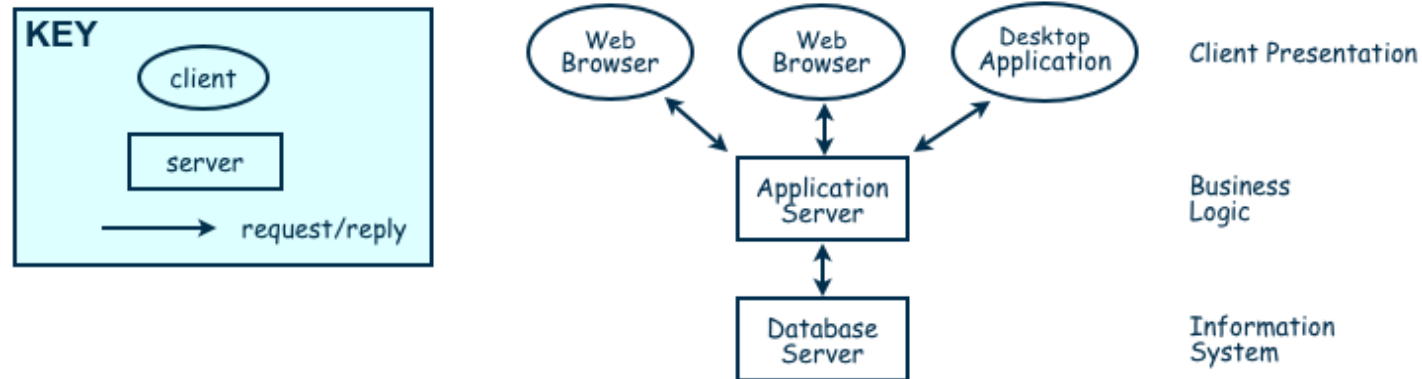
- The system has
 - Streams of data (pipe) for input and output
 - Transformation of the data (filter)



Architectural Styles and Strategies

Client-Server

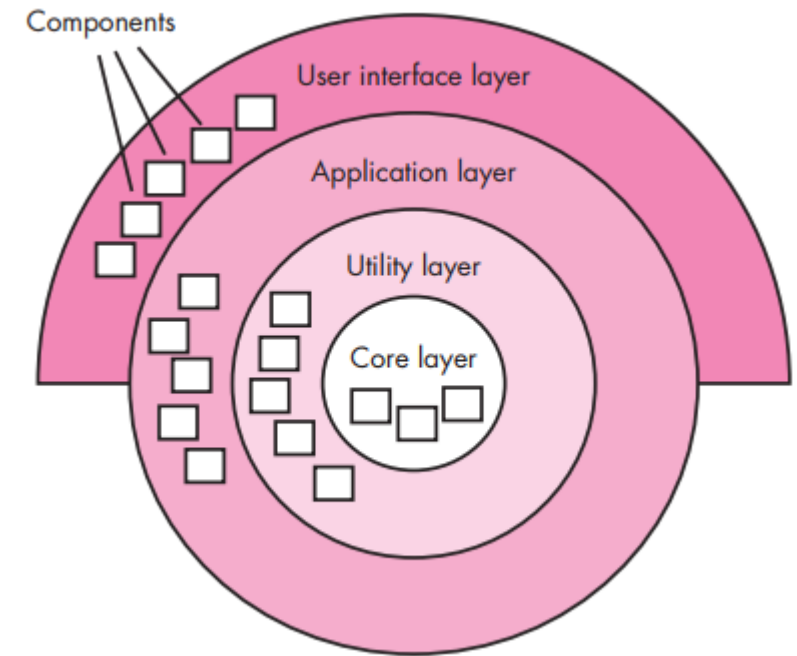
- Two types of components:
 - Server components offer services
 - Clients access them using a request/reply protocol
- Client may send the server an executable function, called a callback
 - The server subsequently calls under specific circumstances



Architectural Styles and Strategies

Layering

- Layers are hierarchical
 - Each layer provides service to the one outside it and acts as a client to the layer inside it
 - Layer bridging: allowing a layer to access the services of layers below its lower neighbor



Architectural Styles and Strategies

Combining Architectural Styles

- Actual software architectures rarely based on purely one style
- Architectural styles can be combined in several ways
 - Use different styles at different layers (e.g., overall client-server architecture with server component decomposed into layers)
 - Use mixture of styles to model different components or types of interaction (e.g., client components interact with one another using publish-subscribe communications)
- If architecture is expressed as collection of models, documentation must be created to show relation between models

Collaborative Design

- Usually the design of software systems is performed by a team of developers
- Several issues must be addressed by the team:
 - Who is best suited to design each aspect of the system
 - How to document all aspects
 - How to coordinate and integrate the software units
- Important to view group interaction in its cultural and ethical contexts

Architecture Evaluation and Refinement

- Design is iterative: we propose design decisions, assess, make adjustments, and propose more decisions
- Many techniques to evaluate the design:
 - Measuring design quality
 - Safety analysis
 - Security analysis
 - Trade-off analysis
 - Cost-benefit analysis
 - Prototyping

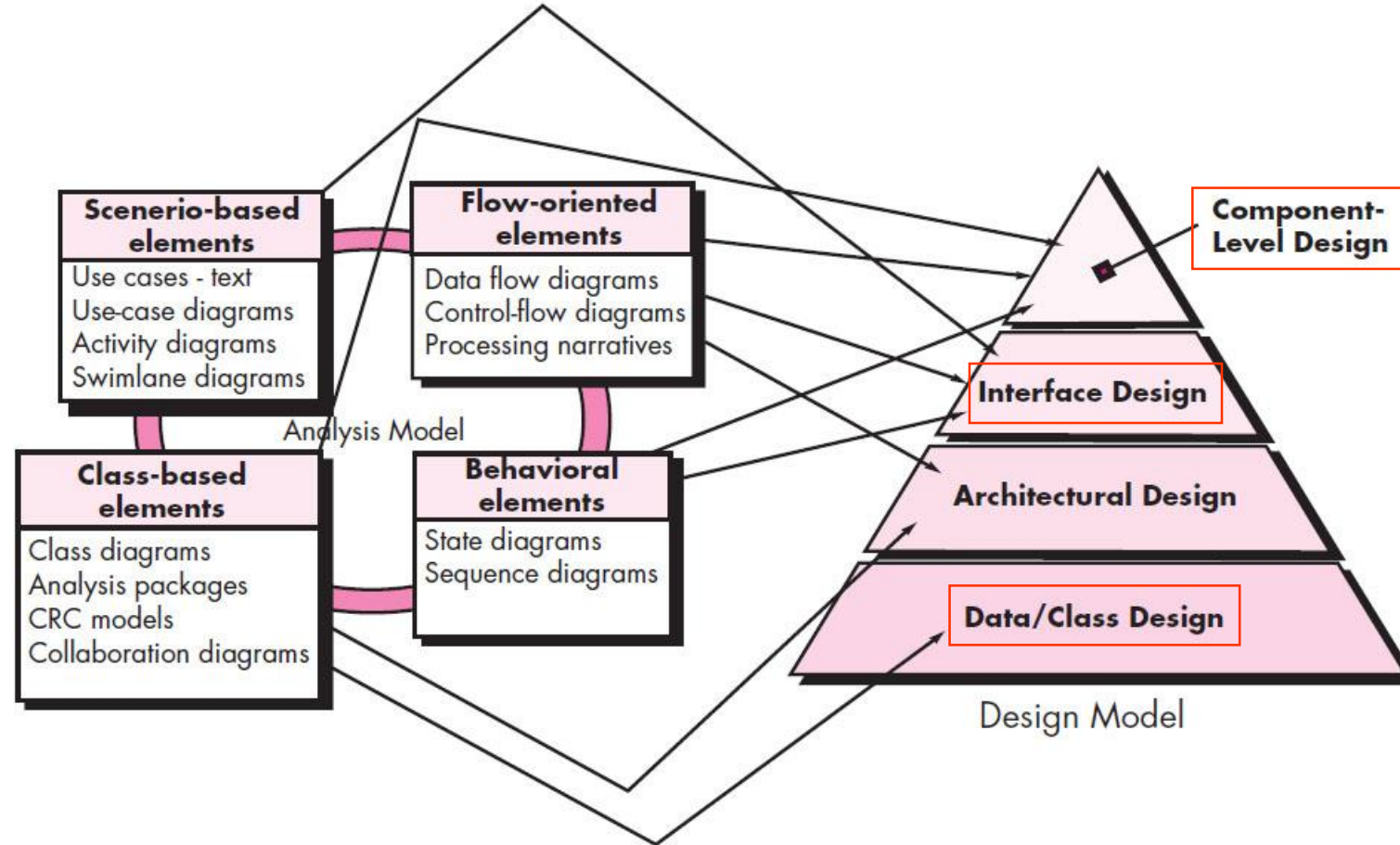
Documenting Software Architectures

- System's architecture is vital to overall development and serves as the basis on decisions for:
 - Design
 - Quality assurance
 - Project management
- The SAD serves as the repository for design information and includes:
 - System overview
 - Views
 - Software units
 - Analysis data and results
 - Design rationale
 - Definitions, glossary, acronyms

Architecture Design Review

- Design review is an essential part of engineering practice
- SAD quality is evaluated in two ways:
 - Validation: making sure the design satisfies all of the customer's requirements (i.e., is this the right system?)
 - Verification: ensuring the design adheres to good design principles (i.e., are we building the system right?)

Content



Content

- Component-Level Design
- Design Principles
- Data Design
- Interface Design
- Graphical Representation of an Algorithm
- Characteristic of Good Design

Component-level design

- The component-level design process encompasses a sequence of activities that slowly reduces the level of abstraction with which software is represented.
- Component-level design ultimately depicts the software at a level of abstraction that is close to code.
- Three different views of component-level design may be taken, depending on the nature of the software to be developed.
 - The object-oriented view
 - The traditional view
 - *A Process-Related View*

Component-level design

The **object-oriented view** focuses on the elaboration of design classes that come from both the problem and infrastructure domain.

- A *component* contains a set of collaborating classes.
- Each *class* within a component has been fully elaborated to include all *attributes* and *operations* that are relevant to its implementation.
- All *interfaces* that enable the classes to communicate and collaborate with other design classes must also be defined.
- The *data structures* appropriate for each attribute must be specified.
- The *algorithmic detail* required to implement the processing logic associated with each operation is designed.

Component-level design

The traditional view

- A component, also called a *module*, resides within the software architecture and serves one of three important roles:
 - A control component* that coordinates the invocation of all other problem domain components
 - A problem domain component* that implements a complete or partial function that is required by the customer
 - An infrastructure component* that is responsible for functions that support the processing required in the problem domain

Component-level design

The traditional view

- A component is a *functional element* of a program that incorporates:
 - Processing logic, the internal *data structures* that are required to implement the processing logic
 - An *interface* that enables the component to be invoked and data to be passed to it. Each data or control object that flows across the interface is represented.
 - The *algorithm* that allows the module to accomplish its intended function is designed using the stepwise refinement approach.

Design Principles

- Design principles are guidelines for decomposing a system's required functionality and behavior into modules
- The principles identify the criteria
 - for decomposing a system
 - deciding what information to provide (and what to conceal) in the resulting modules
- **Six dominant principles:**
 - **Modularity**
 - **Interfaces**
 - Information hiding
 - Incremental development
 - Abstraction
 - Generality

Design Principles

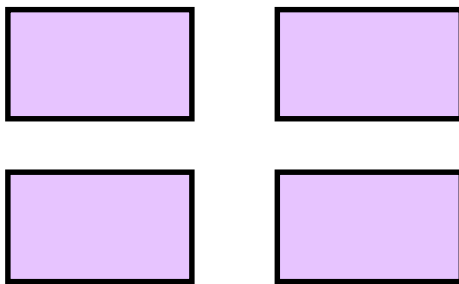
Modularity

- Modularity is the principle of keeping separate the various unrelated aspects of a system, so that each aspect can be studied in isolation (also called separation of concerns)
- If the principle is applied well, each resulting module will have a single purpose and will be relatively independent of the others
 - each module will be easy to understand and develop
 - easier to locate faults (because there are fewer suspect modules per fault)
 - easier to change the system (because a change to one module affects relatively few other modules)
- To determine how well a design separates concerns, we use two concepts that measure module independence: **coupling** and **cohesion**

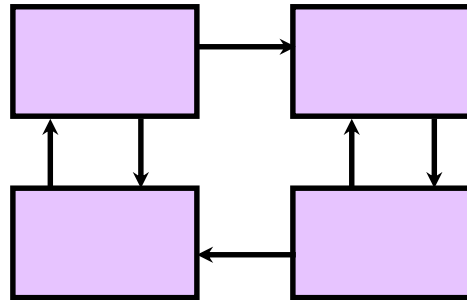
Design Principles

Modularity - Coupling

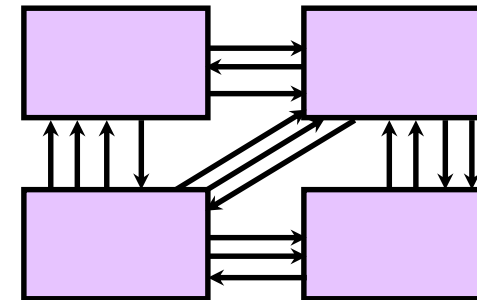
- Two modules are tightly coupled when they depend a great deal on each other
- Loosely coupled modules have some dependence, but their interconnections are weak
- Uncoupled modules have no interconnections at all; they are completely unrelated



Uncoupled -
no dependencies



Loosely coupled -
some dependencies

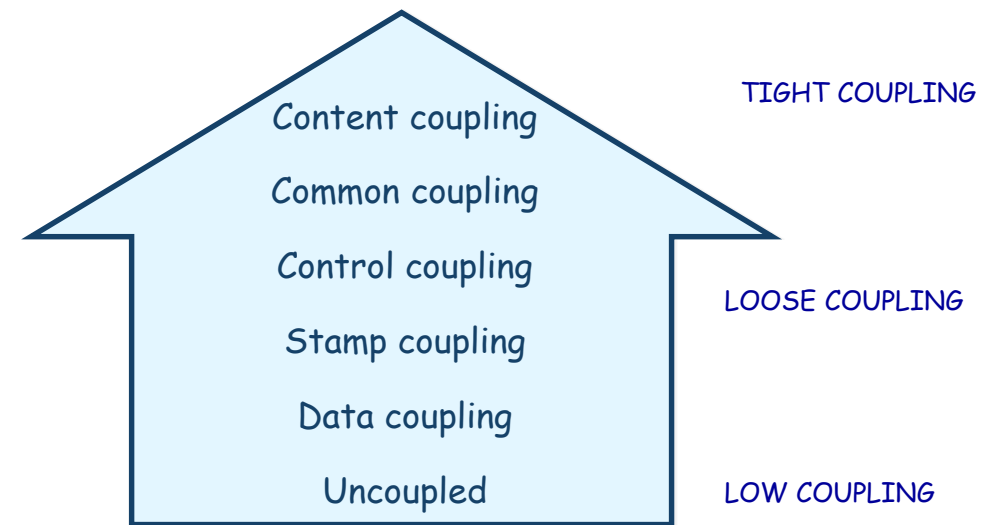


Tightly coupled -
many dependencies

Design Principles

Modularity - Coupling: Types of Coupling

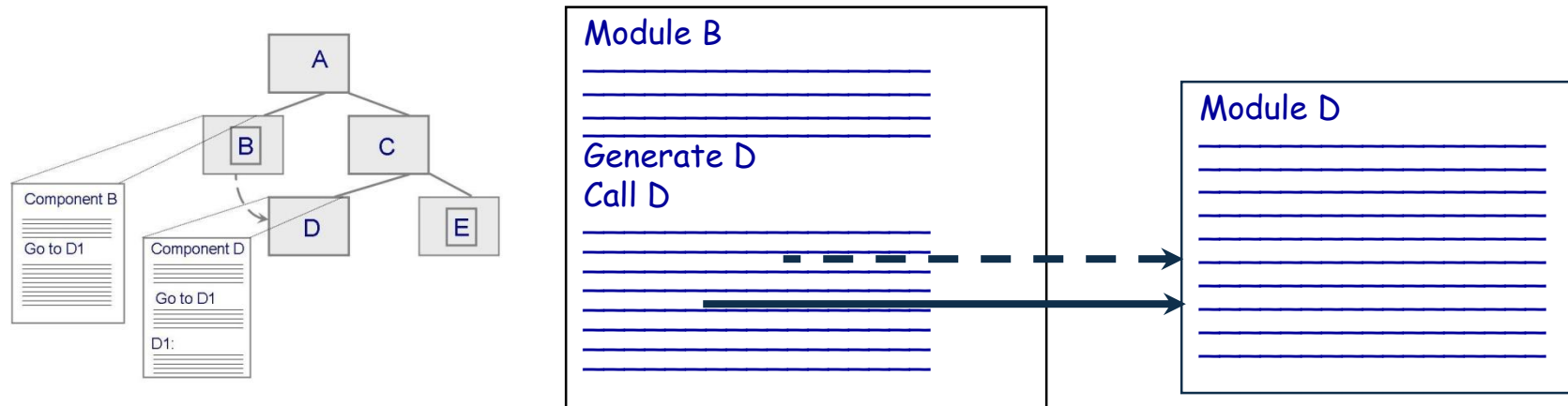
- Content coupling
- Common coupling
- Control coupling
- Stamp coupling
- Data coupling
- Uncoupled



Design Principles

Content Coupling

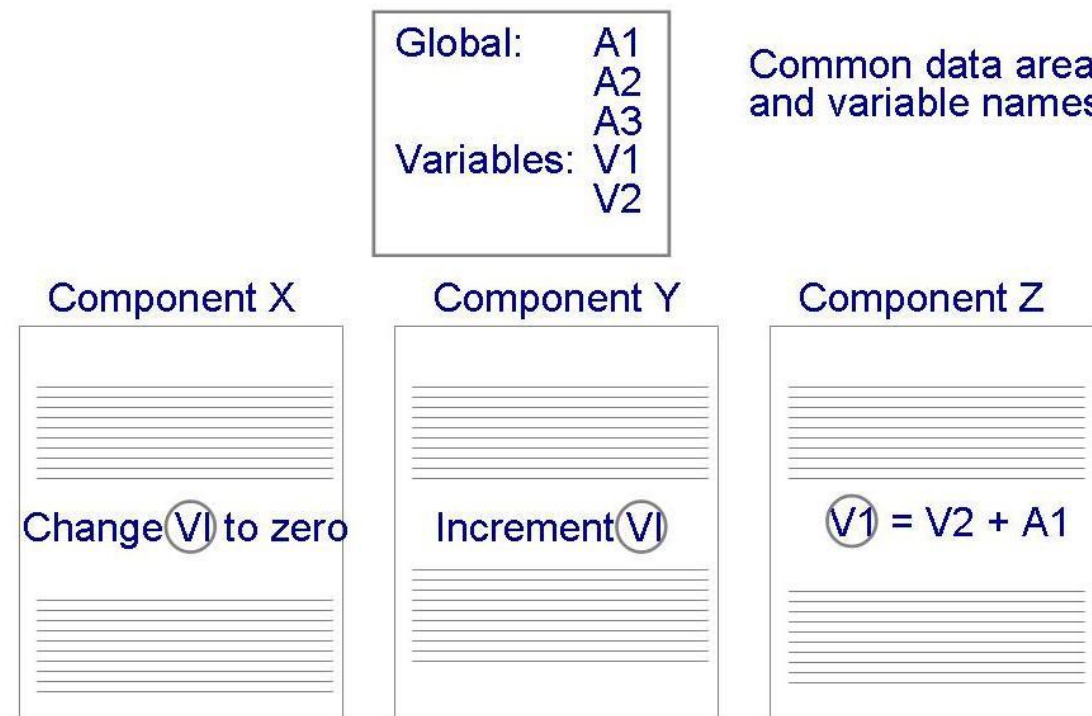
- Occurs when one component modifies an internal data item in another component, or when one component branches into the middle of another component



Design Principles

Common Coupling

- Making a change to the common data means tracing back to all components that access those data to evaluate the effect of the change



Design Principles

Control Coupling

- When one module passes parameters or a return code to control the behavior of another module
- It is impossible for the controlled module to function without some direction from the controlling module

Design Principles

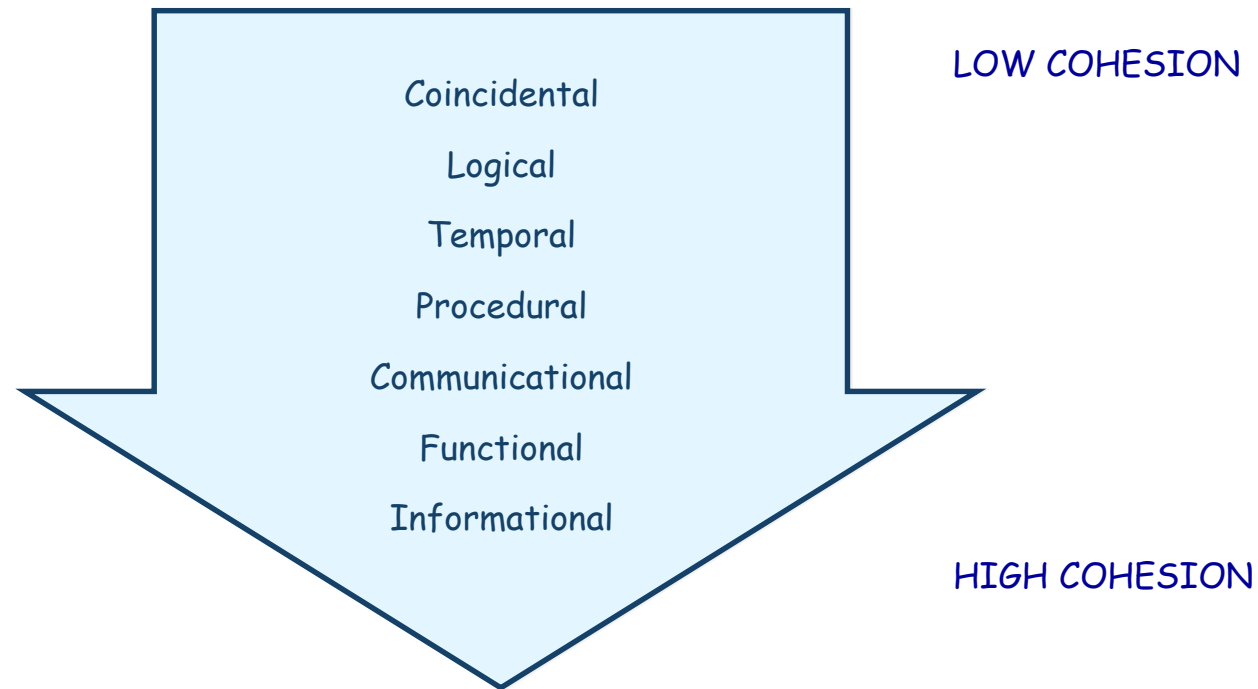
Stamp and Data Coupling

- Stamp coupling occurs when complex data structures are passed between modules
 - Stamp coupling represents a more complex interface between modules, because the modules have to agree on the data's format and organization
- If only data values, and not structured data, are passed, then the modules are connected by data coupling
 - Data coupling is simpler and less likely to be affected by changes in data representation

Design Principles

Modularity - Cohesion

- Cohesion refers to the dependence within and among a module's internal elements (e.g., data, functions, internal modules)



Design Principles

Modularity - Cohesion

- Coincidental (worst degree)
 - Parts are unrelated to one another
- Logical
 - Parts are related only by the logic structure of code
- Temporal
 - Module's data and functions related because they are used at the same time in an execution
- Procedural
 - Similar to temporal, and functions pertain to some related action or purpose

Design Principles

Modularity - Cohesion

- Communication
 - Operates on the same data set
- Functional (ideal degree)
 - All elements essential to a single function are contained in one module, and all of the elements are essential to the performance of the function
- Informational
 - Adaption of functional cohesion to data abstraction and object-based design

Design Principles

Interfaces

- An interface defines what services the software unit provides to the rest of the system, and how other units can access those services
 - For example, the interface to an object is the collection of the object's public operations and the operations' signatures, which specify each operation's name, parameters, and possible return values
- An interface must also define what the unit requires, in terms of services or assumptions, for it to work correctly
- A **software unit's interface** describes what the unit requires of its environment, as well as what it provides to its environment

Design Principles

Information Hiding

- Information hiding is distinguished by its guidance for decomposing a system:
 - Each software unit encapsulates a separate design decision that could be changed in the future
 - Then the interfaces and interface specifications are used to describe each software unit in terms of its externally visible properties
- Using this principle, modules may exhibit different kinds of cohesion
 - A module that hides a data representation may be informationally cohesive
 - A module that hides an algorithm may be functionally cohesive
- A big advantage of information hiding is that the resulting software units are loosely coupled

Design Principles

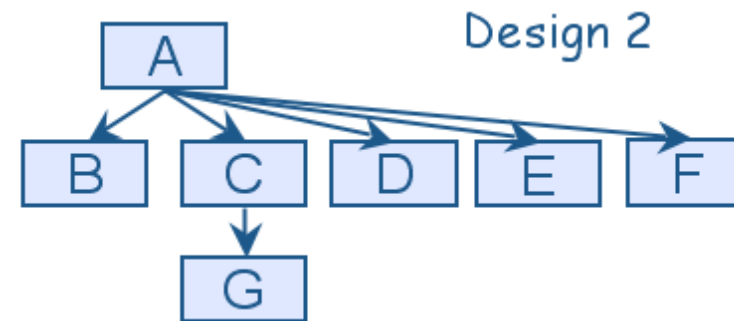
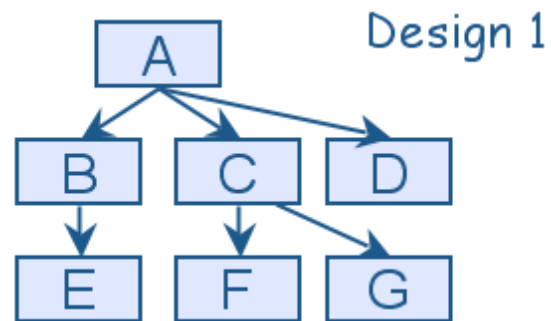
Incremental Development

- Given a design consisting of software units and their interfaces, we can use the information about the units' dependencies to devise an incremental schedule of development
- Start by mapping out the units' uses relation
 - relates each software unit to the other software units on which it depends
- Uses graphs can help to identify progressively larger subsets of our system that we can implement and test incrementally

Design Principles

Incremental Development

- Uses graphs for two designs
 - Fan-in refers to the number of units that use a particular software unit
 - Fan-out refers to the number of units used by particular software unit



Design Principles

Abstraction

- An abstraction is a model or representation that omits some details so that it can focus on other details
- The definition is vague about which details are left out of a model, because different abstractions, built for different purposes, omit different kinds of details

Design Principles

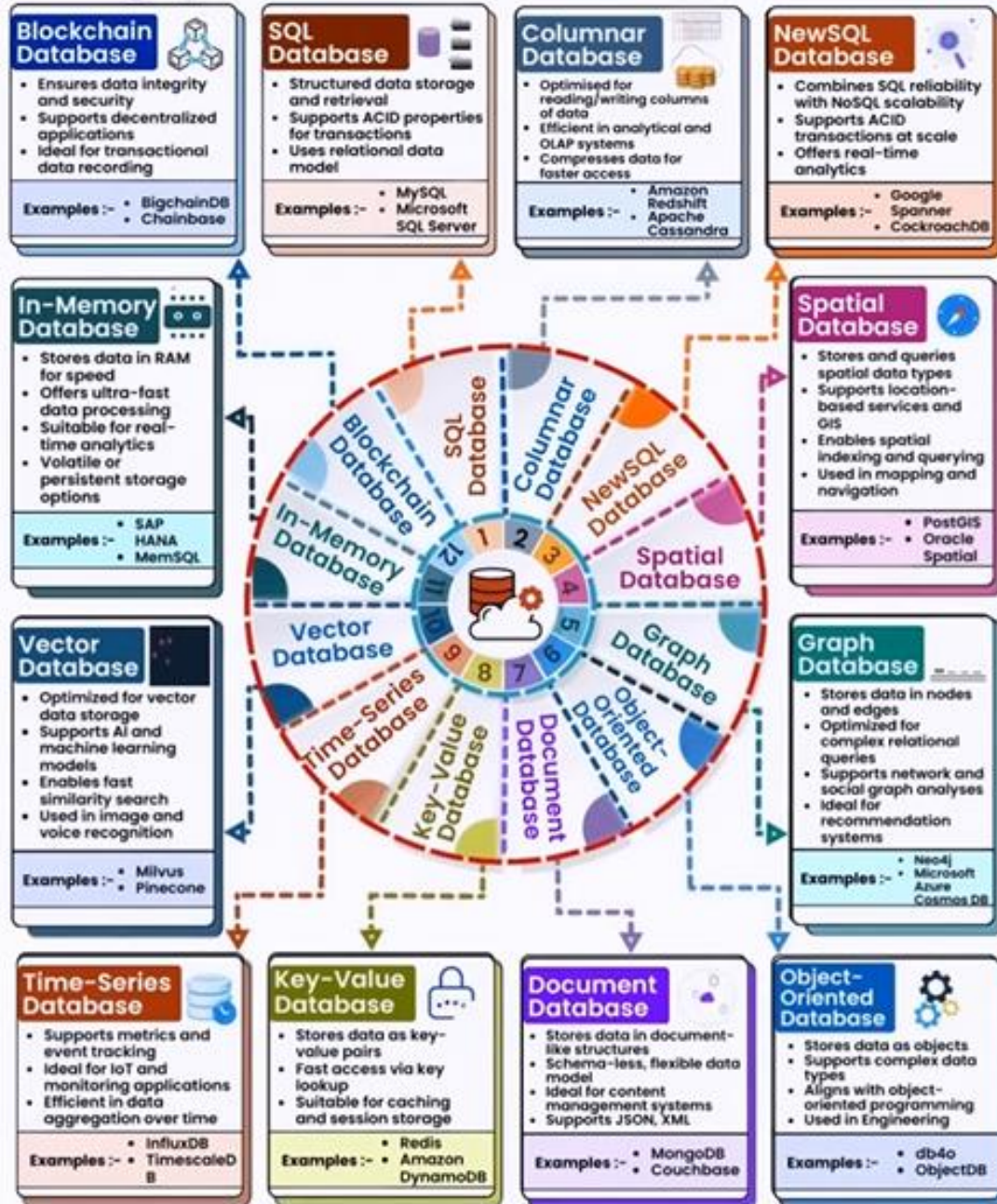
Generality

- Generality is the design principle that makes a software unit as universally applicable as possible, to increase the chance that it will be useful in some future system
- We make a unit more general by increasing the number of contexts in which can it be used. There are several ways of doing this:
 - Parameterizing context-specific information
 - Removing preconditions
 - Simplifying postconditions

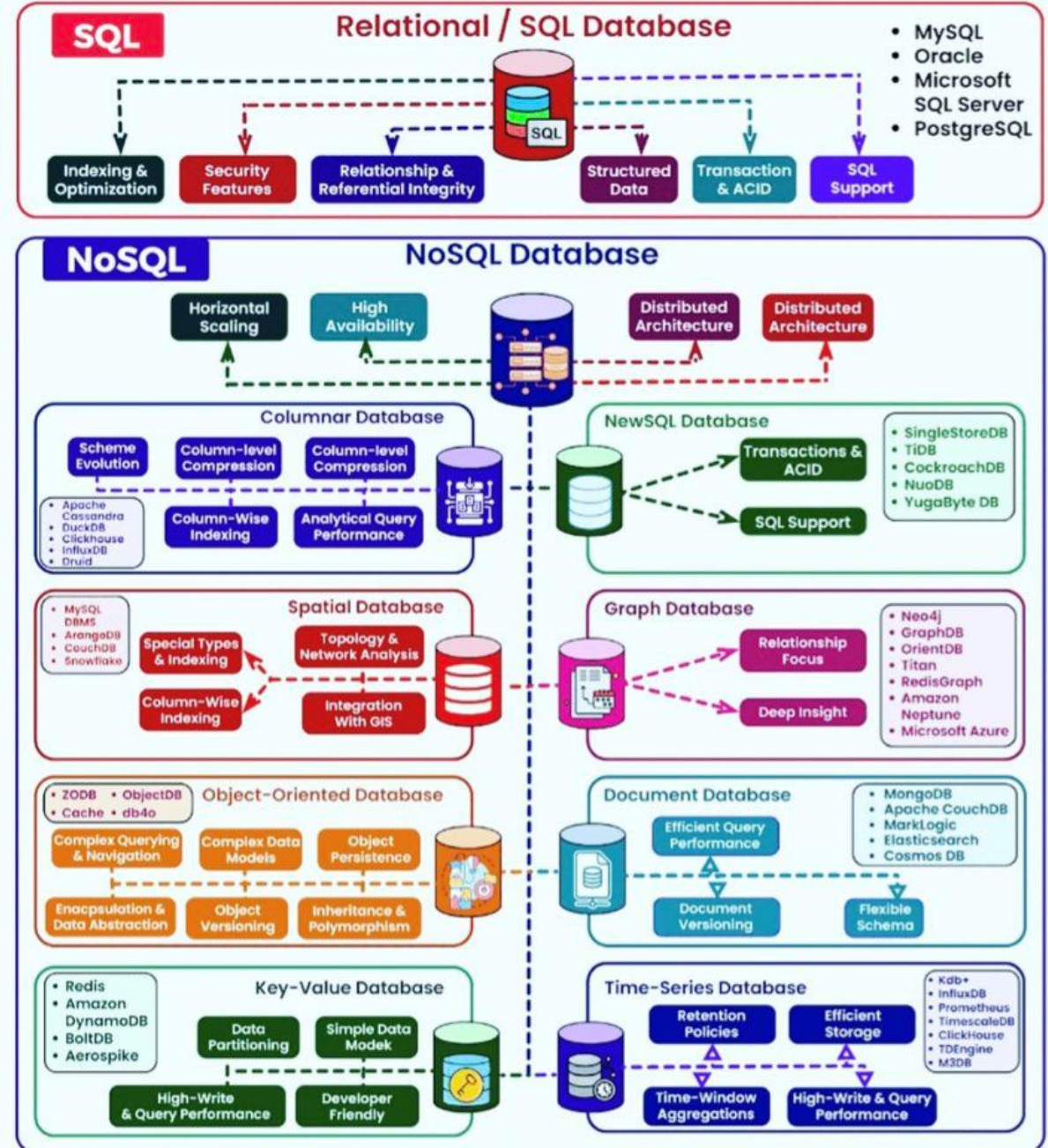
Data Design

- Data design specifies the data structures for implementing the software by converting data objects and their relationships identified during the analysis phase.
- Databases

Types of Databases



Types of Databases



https://www.linkedin.com/posts/brijpandeyji_understanding-database-options-is-key-for-activity-7160957890193707009-Qiq2?utm_source=share&utm_medium=member_desktop

<https://www.linkedin.com/pulse/choosing-right-type-db-important-types-paresh-nayak-i9shf>

Data Design

- Phases of Database Design (*Rational Database*)
 - **Conceptual database design** is to build one (or more) conceptual data models of the data requirements of the enterprise.
 - **Logical database design** results in the creation of a logical data model of the part of the enterprise that we interested in modeling. The conceptual data model created in the previous phase is refined and mapped onto a logical data model.
 - **Physical database design**. It's main aim describe how we intend to physically implement the logical database design. It producing a description of the implementation of the database on secondary storage; it describes the base relations, file organizations, and indexes used to achieve efficient access to the data, and any associated integrity constraints and security measures.

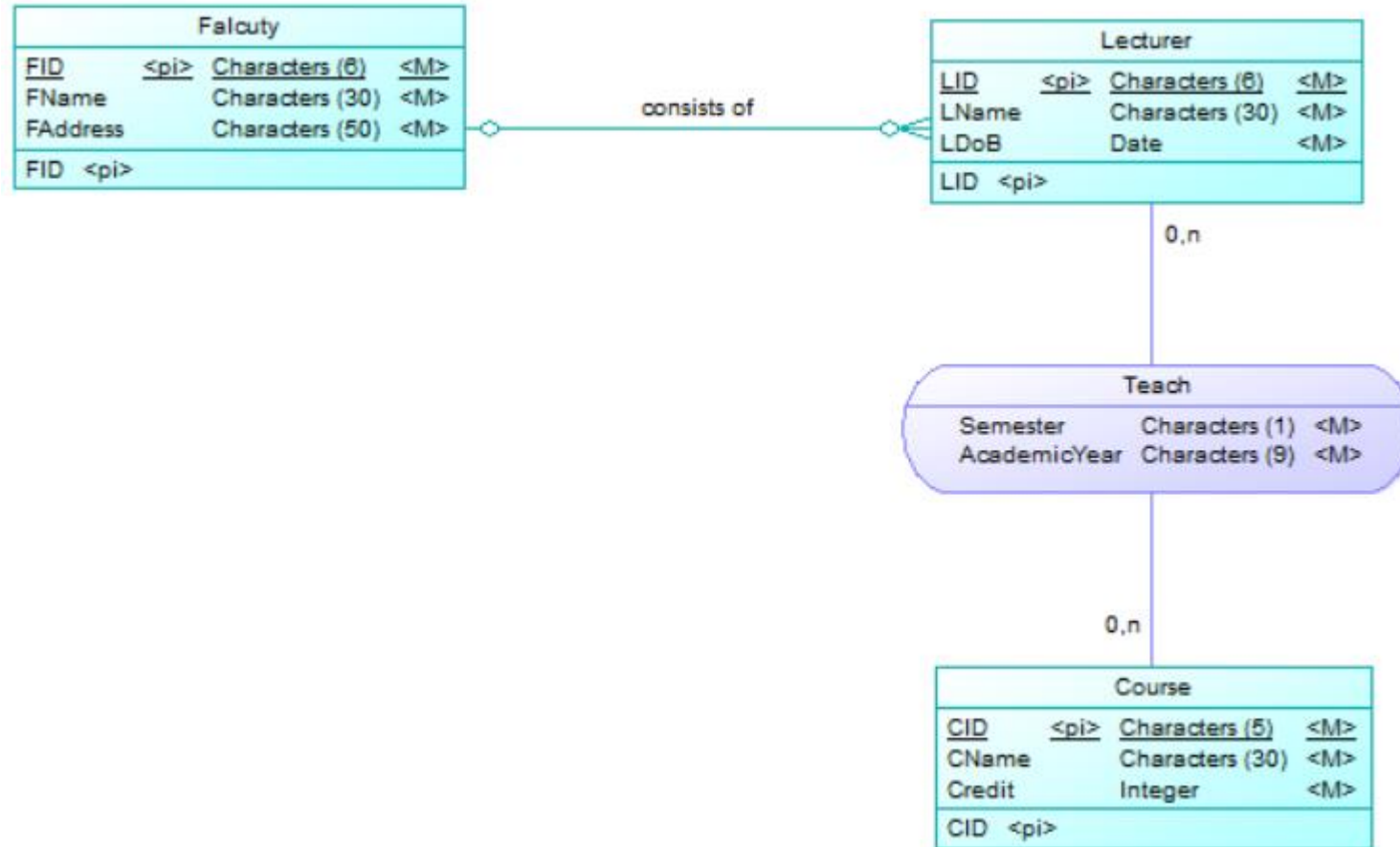
- A **conceptual data model** comprises: entity types; relationship types; attributes and attribute domains; primary keys and alternate keys; integrity constraints.
- *Build Conceptual Data Model*
 - Identify entity types
 - Identify relationship types
 - Identify and associate attributes with entity or relationship types
 - Determine attribute domains
 - Determine candidate, primary, alternate key attributes
 - Consider use of enhanced modeling concepts (optional step)
 - Check model for redundancy
 - Validate conceptual data model against user transactions
 - Review conceptual data model with user

Data Design

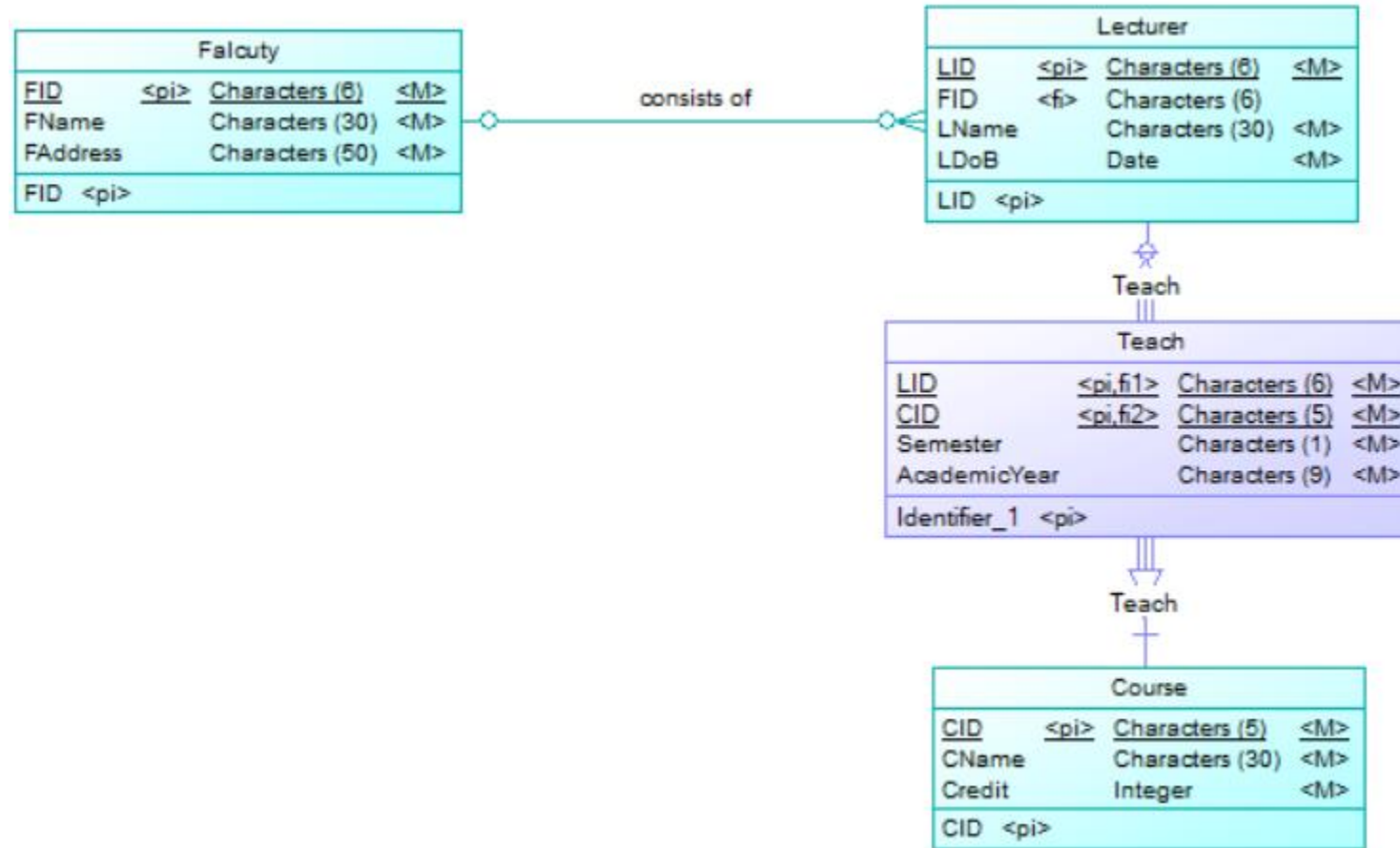
- *Build Logical Data Model:* to translate the conceptual data model into a logical data model and then to validate this model to check that it is structurally correct and able to support the required transactions.
 - Derive relations for logical data model
 - Validate relations using normalization
 - Validate relations against user transactions
 - Check integrity constraints
 - Review logical data model with user
 - Merge logical data models into global data model (optional step)
 - Check for future growth

- The steps of the **physical database design** methodology
 - Translate logical data model for target DBMS
 - Design base relations
 - Design representation of derived data
 - Design general constraints
 - Design file organizations and indexes
 - Analyze transactions
 - Choose file organizations
 - Choose indexes
 - Estimate disk space requirements
 - Design user views
 - Design security mechanisms
 - Consider the introduction of controlled redundancy
 - Monitor and tune the operational system

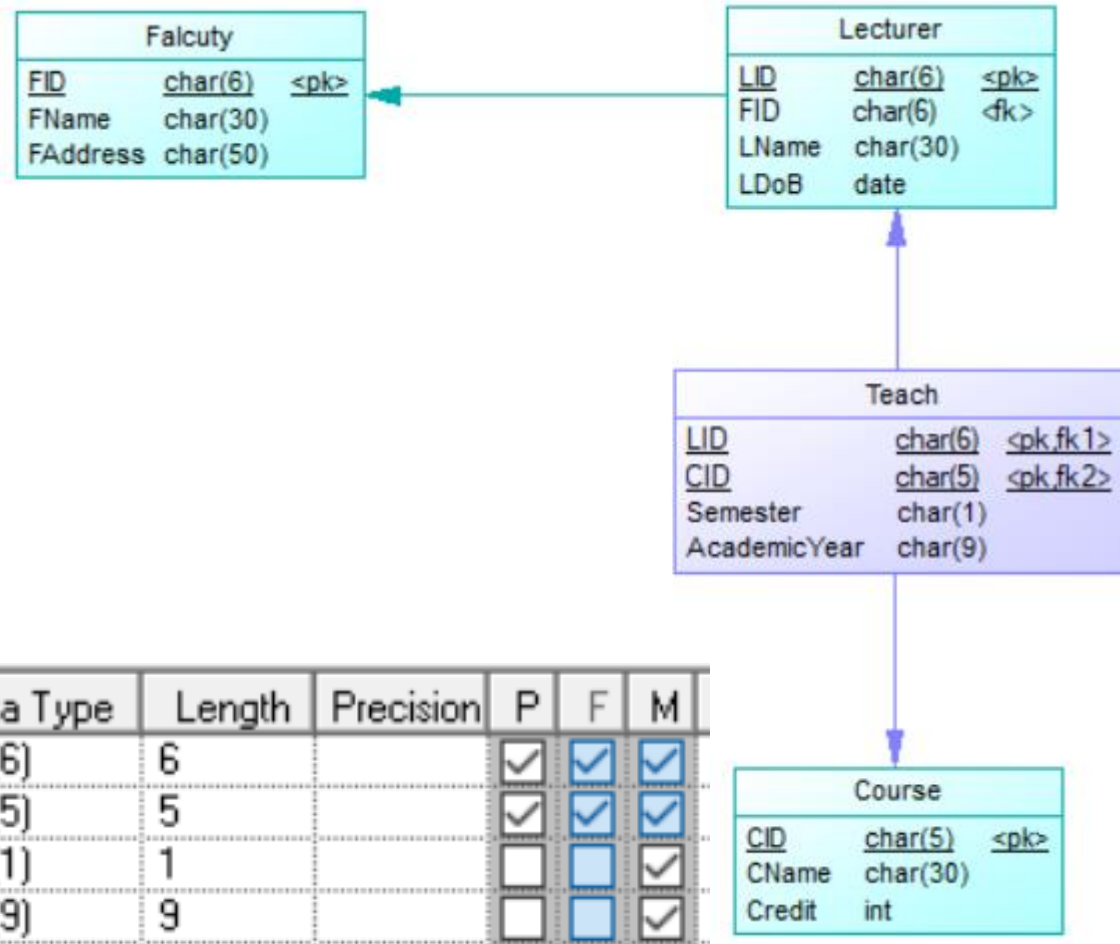
Data Design – Examples



Data Design – Examples



Data Design – Examples



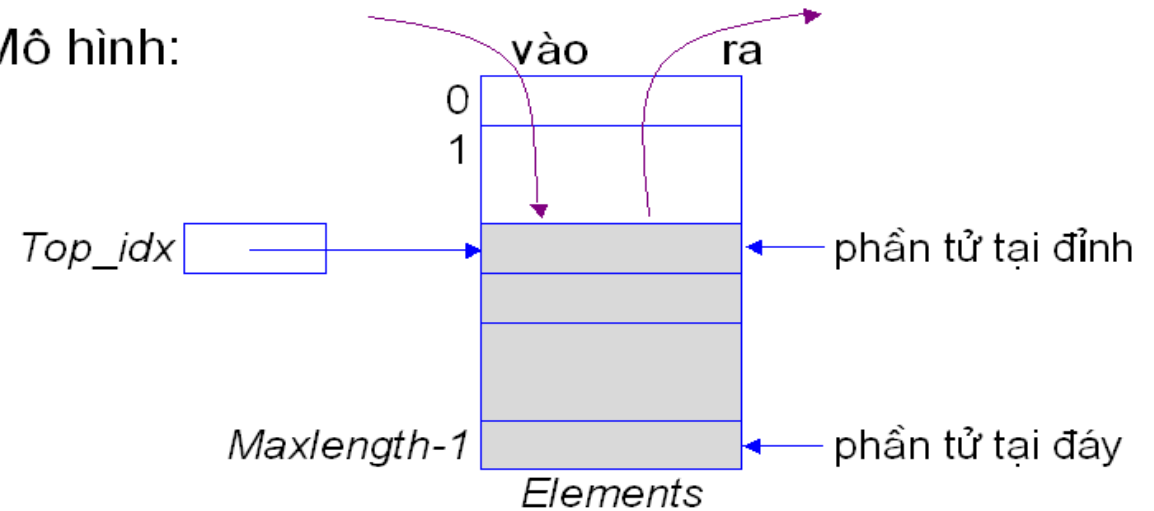
Name	Code	Data Type	Length	Precision	P	F	M
LID	LID	char(6)	6		☒	☒	☒
CID	CID	char(5)	5		☒	☒	☒
Semester	SEMESTER	char(1)	1		☐	☐	☒
AcademicYear	ACADEMICYEA	char(9)	9		☐	☐	☒

Data Design – Examples

- Declaration

```
#define Maxlength      100;
typedef ElementType    float ;
typedef struct {
    ElementType  Elements[Maxlength];
    int          Top_idx;
} Stack;
Stack S;
```

Mô hình:



Interface Design

- Interface design
 - Forms
 - Reports and other outputs of the system

Interface Design – Examples

Teaching Information

Semester: 

Academic Year: 

Lecturer:

001235	Nguyen Van A
001236	Nguyen Thi B
...	...

Course:

CT101	Lap trinh C
CT103	Cau truc du lieu
...	...

Ok

Cancel

Interface Design – Examples

No.	Controller type	Default value	Note
1	Form		
2	Combobox	Current semester	Set: current semester. Values of “Semester” is I, II, III.
3	Combobox	Current academic year	Set: current academic year. Values of “Academic year - ay” is 2 ay before and after the current ay.
4	Combobox		Display LID, LName in table “Lecturer”; select LID
5	Combobox		Display CID, CName in table “Course”; select CID
6	Button		
7	Button		Refresh all comboboxes to the default values.

Teaching Information ①

Semester: ②

Academic Year: ③

Lecturer: ④

001235	Nguyen Van A
001236	Nguyen Thi B
...	...

Course: ⑤

CT101	Lap trinh C
CT103	Cau truc du lieu
...	...

Ok ⑥

Cancel ⑦

No.	Controller type	Default value	Note
1	Form		
2	Combobox	Current semester	Set: current semester. Values of “Semester” is I, II, III.
3	Combobox	Current academic year	Set: current academic year. Values of “Academic year - ay” is 2 ay before and after the current ay.
4	Combobox		Display LID, LName in table “Lecturer”; select LID
5	Combobox		Display CID, CName in table “Course”; select CID
6	Button		
7	Button		Refresh all comboboxes to the default values.

Interface Design – Examples

Teaching Information ①

Semester: ⬇ ② Academic Year: ⬇ ③

Lecturer: ⬇ ④ Course: ⬇ ⑤

001235	Nguyen Van A
001236	Nguyen Thi B
...	...

CT101	Lap trinh C
CT103	Cau truc du lieu
...	...

⑥ ⑦

No.	Table name / Data structure	Method			
		Add	Modify	Delete	Query
1	Teach	x			
2	Lecturer				x
3	Course				x

Interface Design

- The key issues to be considered in interface design:
 - Cultural issues (nationality, ethnicity, gender, occupation, age, region).
 - Interests of users.
- Notes in the interface design:
 - There should be consistency between the interface (menus, commands, display ...).
 - The names of labels are short and easy to remember.
 - Optimizing in dialog box and the mouse movement.

Interface Design

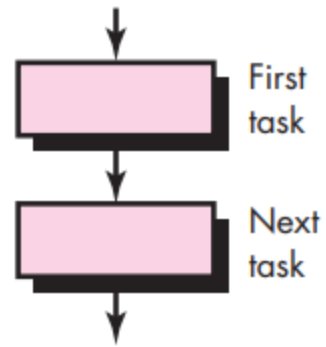
- Notes in the interface design
 - Restricting the direct input data, if possible, users can choose the available data on the combo box.
 - Requesting the confirmation of destructive actions (e.g. erase).
 - Accepting errors from users.
 - Should provide feedback to users.
 - Creating meaningful error messages.
 - Providing the help.

Graphical representation of an algorithm

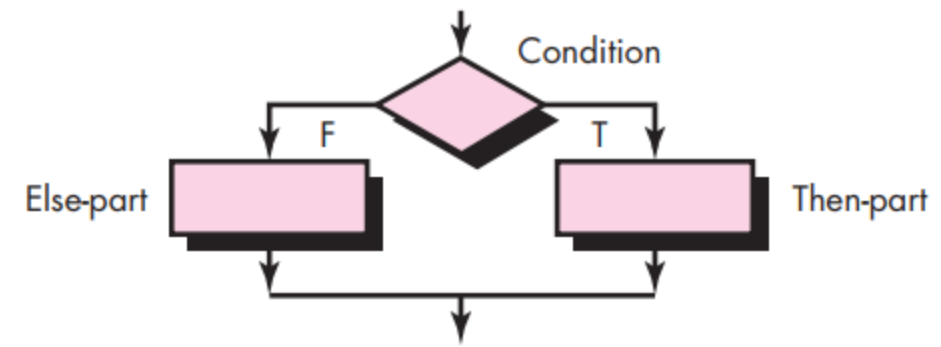
- Algorithm is an ordered set of unambiguous steps that produces a desired output from a given input and terminates in a finite time.
- Presentation methods
 - Flowchart/ UML activity diagram.
 - Pseudocode text
- A **flowchart** is a graphical representation of an algorithm.

Symbol Name	Symbol	function
Oval		Used to represent start and end of flowchart
Parallelogram		Used for input and output operation
Rectangle		Processing: Used for arithmetic operations and data-manipulations
Diamond		Decision making. Used to represent the operation in which there are two/three alternatives, true and false etc
Arrows		Flow line Used to indicate the flow of logic by connecting symbols
Circle		Page Connector

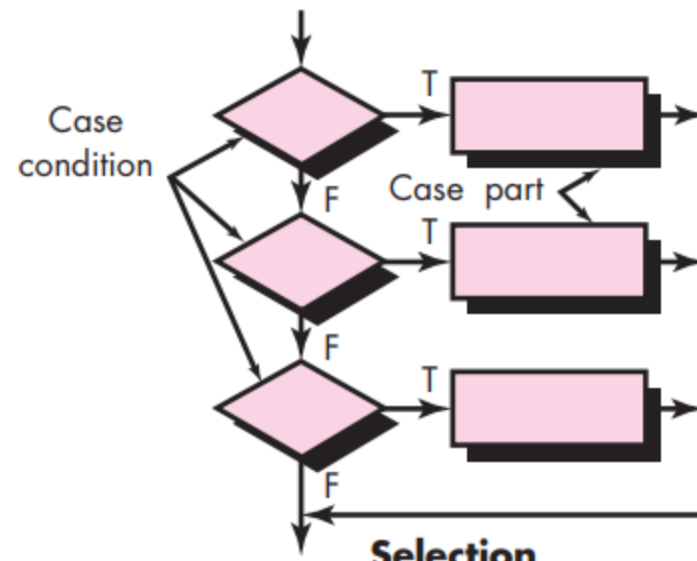
	Off Page Connector
	Predefined Process /Function Used to represent a group of statements performing one processing task.
	Preprocessor
	Comments



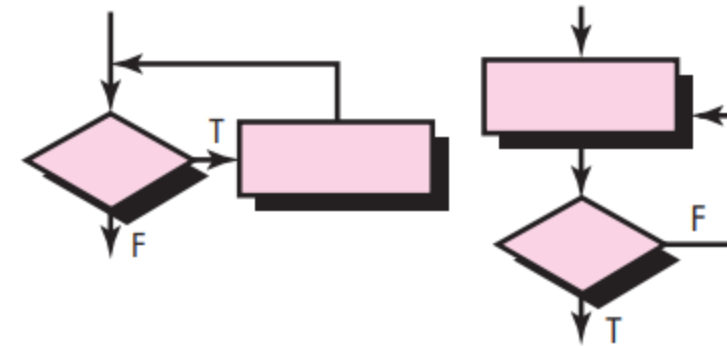
Sequence



If-then-else



Selection



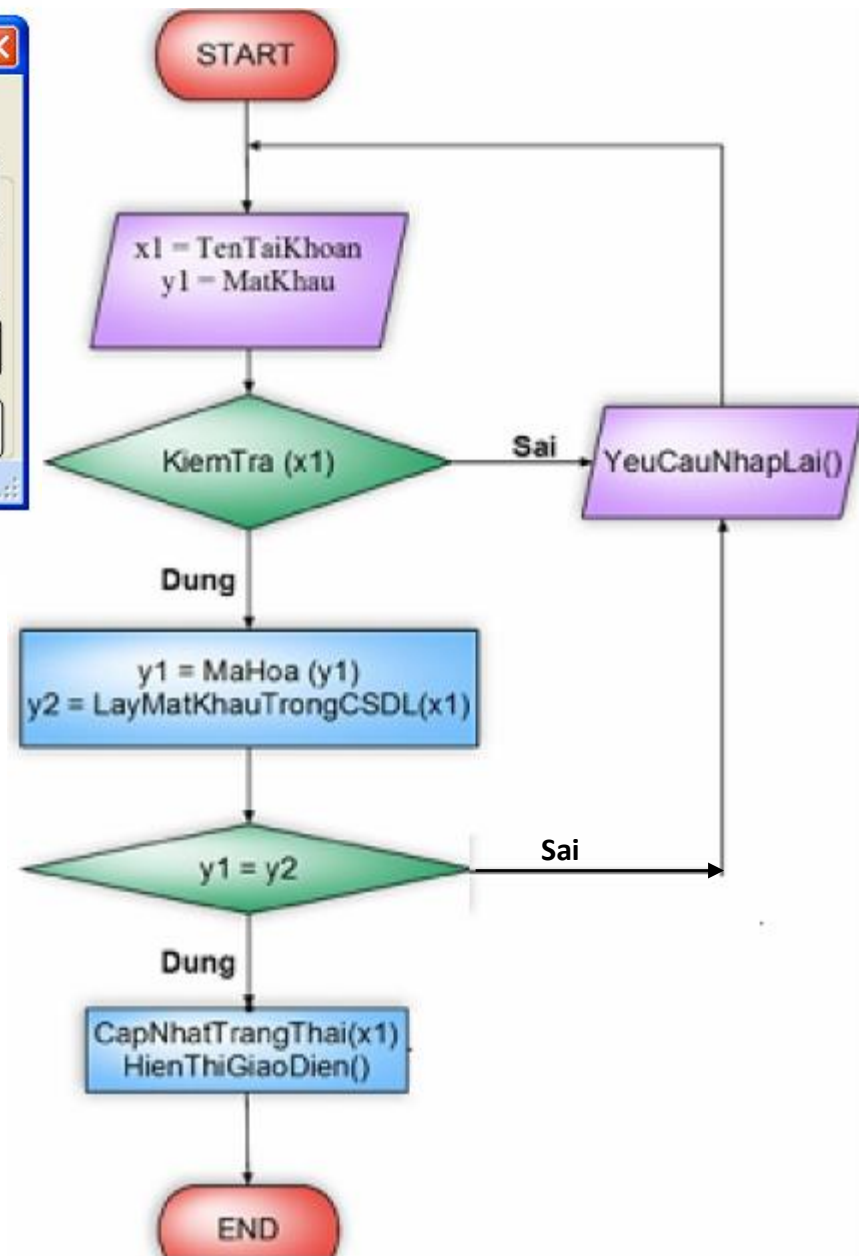
Do while

Repeat until

Repetition



Hình 1: Giao diện Đăng Nhập



The flowchart of “Đăng Nhập”

Flowchart – Examples

Characteristics of Good Design

- Component independence
 - coupling
 - cohesion
- Exception identification and handling
- Fault prevention and tolerance
 - active
 - passive

Q&A

References

- Shari Lawrence Pfleeger, Joanne M. Atlee, Software Engineering theory and practice, Pearson Prentice Hall, 4th edition, 2010.
- Roger S. Pressman, Software Engineering: A Practitioner's Approach - 7th edition, McGraw-Hill, 2009.
- Thomas Connolly and Carolyn Begg, Database Systems: A Practical Approach to Design, Implementation, and Management - 6th edition, Pearson Education Limited, 2015.
- Hiep Xuan Huynh, Lan Phuong Phan, Introduction to Software Engineering, Can Tho university publishing house, 2010.